

Got it  — you want a **simple Linux Makefile** that compiles **main.cpp** and links it against an **existing .so shared library**.

Here's a minimal but robust example:

Makefile

```
# Compiler and flags
CXX      := g++
CXXFLAGS := -Wall -Wextra -O2 -std=c++17

# Paths to library and includes
LIB_DIR  := /path/to/lib          # directory containing libmylib.so
INC_DIR  := /path/to/include      # directory containing library headers

# Library name without 'lib' prefix and '.so' suffix
LIB_NAME := mylib

# Target executable
TARGET   := main

# Source files
SRC      := main.cpp

# Build rule
all: $(TARGET)

$(TARGET): $(SRC)
```

```

$(CXX) $(CXXFLAGS) -I$(INC_DIR) -L$(LIB_DIR) -l$(LIB_NAME) $< -o $@

# Run the program (sets LD_LIBRARY_PATH so it finds the .so)
run: $(TARGET)
    LD_LIBRARY_PATH=$(LIB_DIR):$$LD_LIBRARY_PATH ./$(TARGET)

# Clean build artifacts
clean:
    rm -f $(TARGET)

.PHONY: all run clean

```

How to use

1. Adjust paths:

1. Set **LIB_DIR** to the folder containing your **.so** file (e.g., **/usr/local/lib** or **./lib**).
2. Set **INC_DIR** to the folder containing the library's **.h** files.
3. Set **LIB_NAME** to the library name without **lib** prefix or **.so** suffix.
Example: **libmylib.so** → **mylib**.

2. Build:

```
make
```

3. Run (ensures **.so** is found at runtime):

```
make run
```

4. Clean:

```
make clean
```



Tip:

If your **.so** is in a non-standard location and you don't want to set **LD_LIBRARY_PATH** every time, you can embed the runtime path into the binary:

```
$(CXX) $(CXXFLAGS) -I$(INC_DIR) -L$(LIB_DIR) -Wl,-rpath,$(LIB_DIR) -l$(LIB_NAME) $< -o $@
```

If you want, I can also give you a **version that automatically detects .so files in a lib/ folder** so you don't have to hardcode the library name.

Do you want me to prepare that?